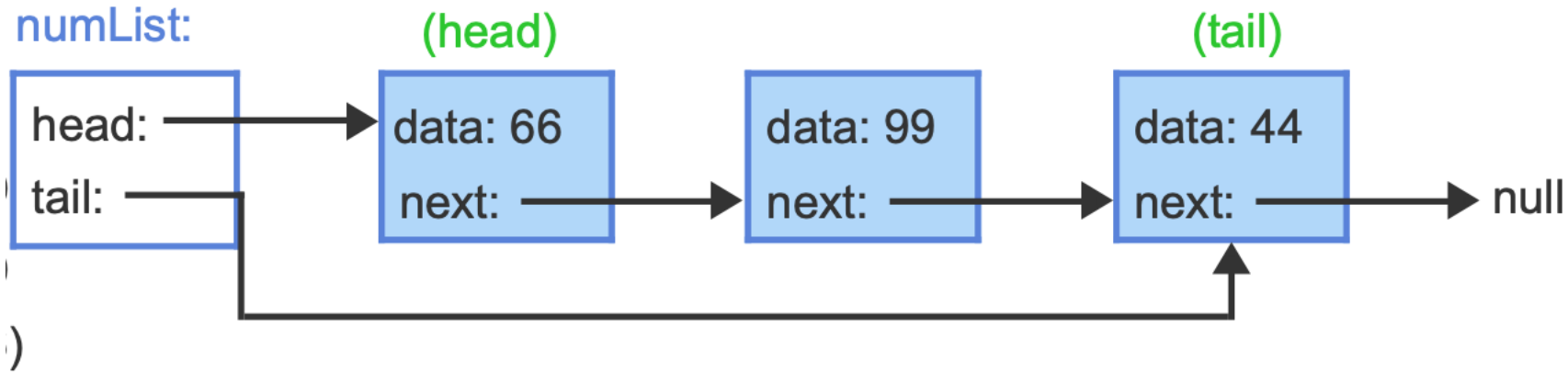
List, Queues, Stacks, Deque and Bag

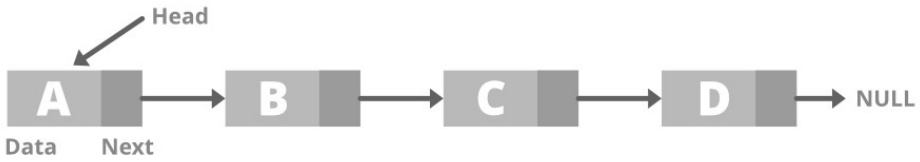
Linked List Terms

- **Node** – each node of a linked list can store a piece of **data**.
- **Next** – each node of a linked list contains *the next node* called **next**.
- **Head** – first node of the linked list.
- **Tail** – the last node of a linked list

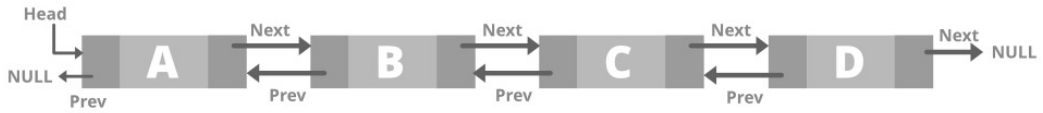


Types of linked list

Singly Linked List



Doubly Linked List



Circular Linked List



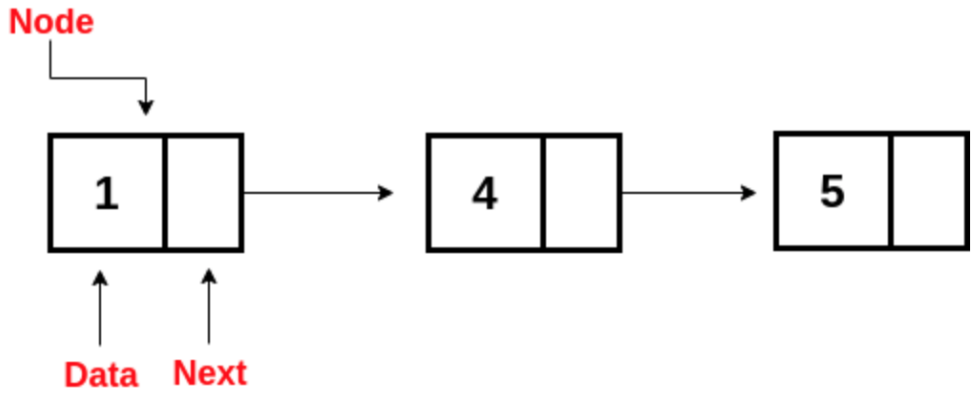
Singly Linked List

A Singly-linked list is a collection of nodes linked together in a sequential way where each node of the singly linked list contains a data field and an address field that contains the reference of the next node.

The structure of the node in the Singly Linked List is

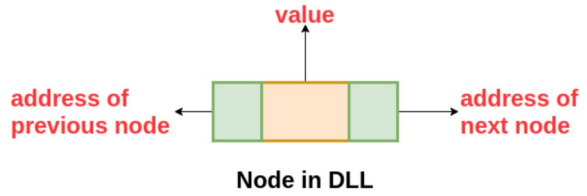
```
class Node {
    int data // variable to store the data of the node
    Node next // variable to store the address of the next node
}
```

The nodes are connected to each other in this form where the value of the next variable of the last node is NULL i.e. ***next*** = ***NULL***, which indicates the end of the linked list.



Doubly Linked List

A Doubly Linked List contains an extra memory to store the address of the previous node, together with the address of the next node and data which are there in the singly linked list. So, here we are storing the address of the next as well as the previous nodes.




```
class DLLNode {
    int val // variable to store the data of the node
    DLLNode prev // variable to store the address of the previous node
    DLLNode next // variable to store the address of the next node
}
```

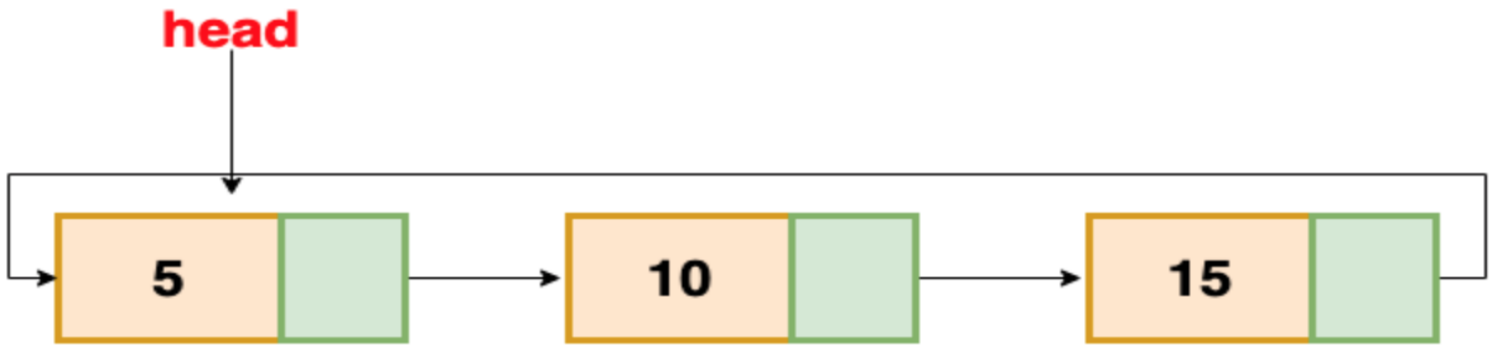
The nodes are connected to each other in this form where the first node has **prev = NULL** and the last node has **next = NULL**.



Doubly Linked List

Circular Linked List

A circular linked list is either a singly or doubly linked list in which there are no ***NULL*** values. Here, we can implement the Circular Linked List by making the use of Singly or Doubly Linked List. In the case of a singly linked list, the next of the last node contains the address of the first node and in case of a doubly-linked list, the next of last node contains the address of the first node and prev of the first node contains the address of the last node.



Circular Linked List

Basic Operations on Linked List

- **Traversal**: To traverse all the nodes one after another.
- **Insertion**: To add a node at the given position.
- **Deletion**: To delete a node.
- **Searching**: To search an element(s) by value.
- **Updating**: To update a node.
- **Sorting:** To arrange nodes in a linked list in a specific order.
- **Merging:** To merge two linked lists into one.

Linked List Traversal

The idea here is to step through the list from beginning to end. ***For example***, we may want to print the list or search for a specific node in the list.

The algorithm for traversing a list

- Start with the head of the list. Access the content of the head node if it is not null.
- Then go to the next node(if exists) and access the node information
- Continue until no more nodes (that is, you have reached the null node)

```
void traverseLL(Node head) {
    while(head != NULL)
    {
        print(head.data)
        head = head.next
    }
}
```

Linked List Insert

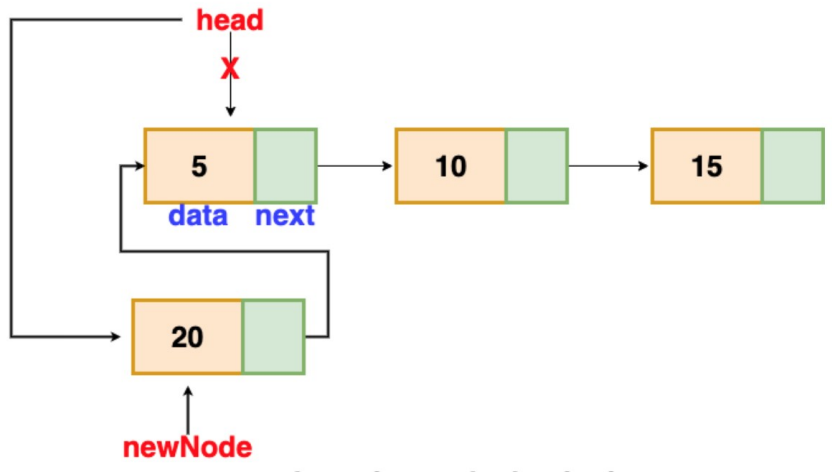
There can be three cases that will occur when we are inserting a node in a linked list.

- Insertion at the beginning
- Insertion at the end. (Append)
- Insertion after a given node

Insert at beginning

Since there is no need to find the end of the list. If the list is empty, we make the new node as the head of the list. Otherwise, we we have to connect the new node to the current head of the list and make the new node, the head of the list.

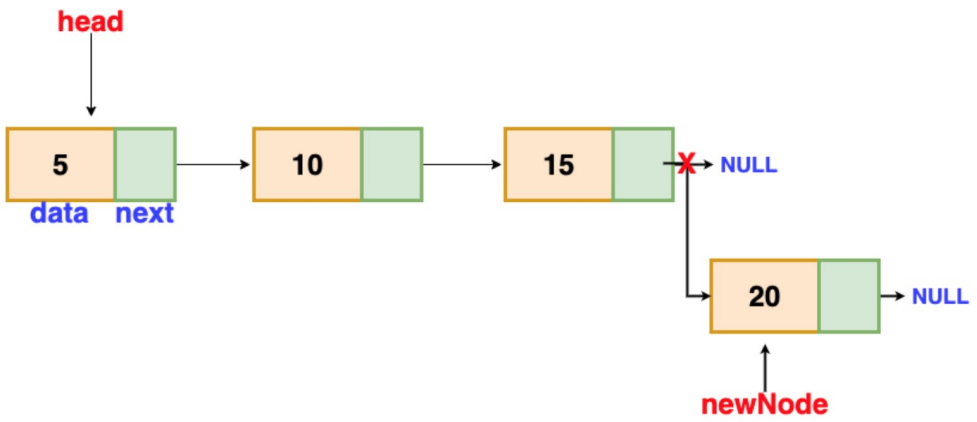
```
// function is returning the head of the singly linked-list
Node insertAtBegin(Node head, int val)
{
    newNode = new Node(val) // creating new node of linked list
    if(head == NULL) // check if linked list is empty
        return newNode
    else // inserting the node at the beginning
    {
        newNode.next = head
        return newNode
    }
}
```



Insert at End

We will traverse the list until we find the last node. Then we insert the new node to the end of the list. Note that we have to consider special cases such as list being empty.
In case of a list being empty, we will return the updated head of the linked list because in this case, the inserted node is the first as well as the last node of the linked list.

```
// the function is returning the head of the singly linked list
Node insertAtEnd(Node head, int val)
{
    if( head == NULL ) // handing the special case
    {
        newNode = new Node(val)
        head = newNode
        return head
    }
    Node temp = head
    // traversing the list to get the last node
    while( temp.next != NULL )
    {
        temp = temp.next
    }
    newNode = new Node(val)
    temp.next = newNode
    return head
}
```

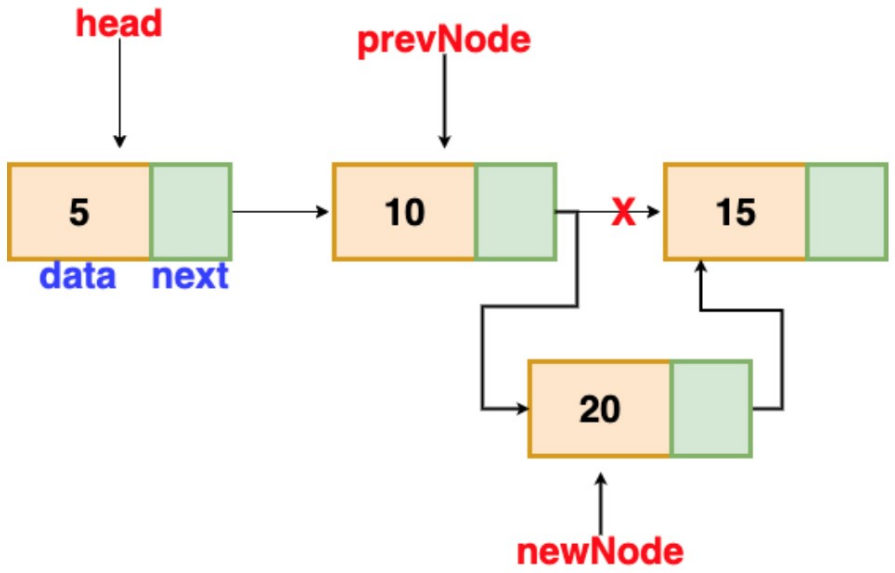


Insert after/before a given node

We are given the reference to a node, and the new node is inserted after/before the given node.

```
void insertAfter(Node prevNode, int val)
{
    newNode = new Node(val)

    newNode.next = prevNode.next
    prevNode.next = newNode
}
```

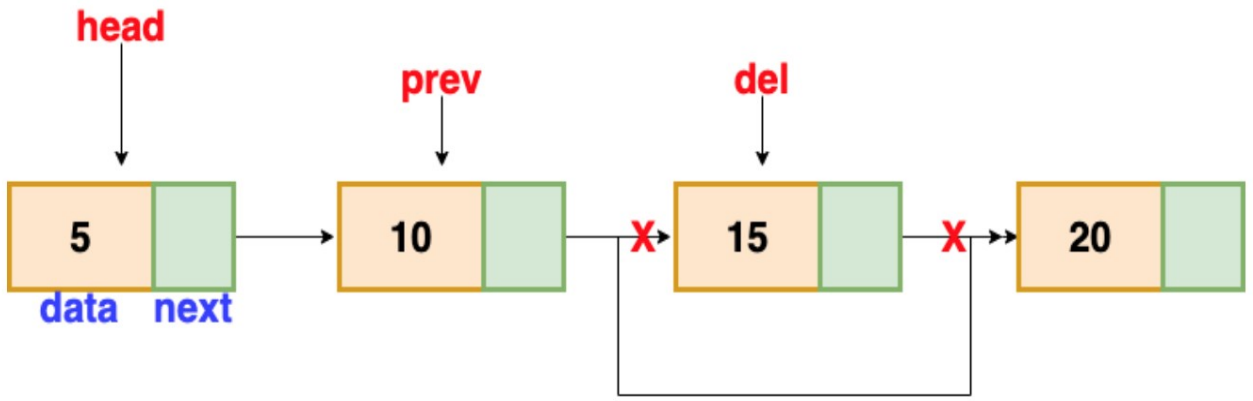


Linked List Delete

To delete a node from a linked list, we need to do these steps

- Find the previous node of the node to be deleted.
- Change the next pointer of the previous node
- Free the memory of the deleted node.

In the deletion, there is a special case in which the first node is deleted. In this, we need to update the head of the linked list.




```
// this function will return the head of the linked list
Node deleteLL(Node head, Node del)
{
    if(head == del) // if the node to be deleted is the head node
    {
        return head.next // special case for the first Node
    }
    Node temp = head

    while( temp.next != NULL )
    {
        if(temp.next == del) // finding the node to be deleted
        {
            temp.next = temp.next.next
            delete del // free the memory of that Node
            return head
        }
        temp = temp.next
    }
    return head // if no node matches in the Linked List
}
```

Linked List Searching

To search any value in the linked list, we can traverse the linked list and compares the value present in the node.

```
bool searchLL(Node head, int val)
{
    Node temp = head // creating a temp variable pointing to the head of the l
inked list
    while( temp != NULL) // traversing the list
    {
        if( temp.data == val )
            return true
        temp = temp.next
    }
    return false
}
```

The above searching code is an example of linear search.

Linked List Updating

To update the value of the node, we just need to set the data part to the new value.

Below is the implementation in which we had to update the value of the first node in which data is equal to **val** and we have to set it to **newVal**.

```
void updateLL(Node head, int val, int newVal)
{
    Node temp = head
    while(temp != NULL)
    {
        if( temp.data == val)
        {
            temp.data = newVal
            return
        }
        temp = temp.next
    }
}
```

Linked List Updating

To update the value of the node, we just need to set the data part to the new value.

Below is the implementation in which we had to update the value of the first node in which data is equal to **val** and we have to set it to **newVal**.

```
void updateLL(Node head, int val, int newVal)
{
    Node temp = head
    while(temp != NULL)
    {
        if( temp.data == val)
        {
            temp.data = newVal
            return
        }
        temp = temp.next
    }
}
```

Stacks

- A stack is a container of objects that are inserted and removed according to the last-in-first-out (LIFO) principle.
- Objects can be inserted at any time, but only the last (the most-recently inserted) object can be removed.
- Inserting an item is known as “pushing” onto the stack. “Popping” off the stack is synonymous with removing an item.

Stack Methods

Operation	Description	Example starting with stack: 99, 77 (top is 99).
Push(stack, x)	Inserts x on top of stack	Push(stack, 44). Stack: 44, 99, 77
Pop(stack)	Returns and removes item at top of stack	Pop(stack) returns: 99. Stack: 77
Peek(stack)	Returns but does not remove item at top of stack	Peek(stack) returns 99. Stack still: 99, 77
IsEmpty(stack)	Returns true if stack has no items	IsEmpty(stack) returns false.
GetLength(stack)	Returns the number of items in the stack	GetLength(stack) returns 2.

Stack Implementation

While, the stack data structure is a "built-in" class of Java's java.util package, it is possible, and sometimes preferable to define your own specific one, like this:

```
public interface Stack {

    // accessor methods

  public int size(); // return the number of
                     // elements in the stack

  public boolean isEmpty(); // see if the stack
                            // is empty

  public Object top() // return the top element

    throws StackEmptyException; // if called on
                                // an empty stack

                                // update methods


  public void push (Object element); // push an
                     // element onto the stack

  public Object pop() // return and remove the
                     // top element of the stack

    throws StackEmptyException; // if called on
                                // an empty stack

}
```

Stacks and Lists

Linked List:

Push is accomplished using Insert Before (Prepend)
Pop is accomplish using List Delete
Size is accomplished using List Traversal
peek is accomplished by returning the first node
isEmpty() is accomplished by checking if head is null.

Array List:

Push is accomplished adding shifting all the elements to the right. Allocate more storage if necessary.
Pop is accomplished by shifting all elements left.
Size is accomplished using List Traversal
peek is accomplished by returning the first value in the array
isEmpty() is accomplished by checking if head is null.

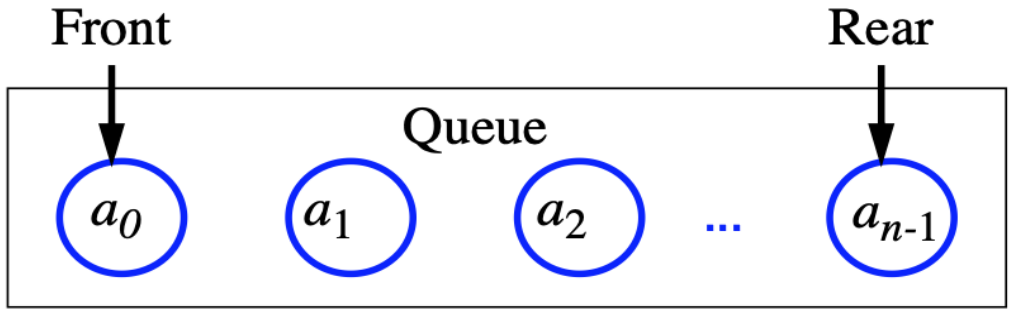
Array based lists

An array-based list supports the common list ADT operations, such as append, prepend, insert after, remove, and search.



Queue

- A queue differs from a stack in that its insertion and removal routines follows the first-in-first-out (FIFO) principle.
- Elements may be inserted at any time, but only the element which has been in the queue the longest may be removed.
- Elements are inserted at the *rear* (enqueued) and removed from the *front* (dequeued)



Queue Methods

Operation	Description	Example starting with queue: 43, 12, 77 (front is 43)
Enqueue(queue, x)	Inserts x at end of the queue	Enqueue(queue, 56). Queue: 43, 12, 77, 56
Dequeue(queue)	Returns and removes item at front of queue	Dequeue(queue) returns: 43. Queue: 12, 77
Peek(queue)	Returns but does not remove item at the front of the queue	Peek(queue) return 43. Queue: 43, 12, 77
IsEmpty(queue)	Returns true if queue has no items	IsEmpty(queue) returns false.
GetLength(queue)	Returns the number of items in the queue	GetLength(queue) returns 3.

Deque

The *Java Deque* interface, java.util.Deque, represents a double ended queue, meaning a queue where you can add and remove elements to and from both ends of the queue. The name *Deque* is an abbreviation of *Double Ended Queue*. The word Deque is pronounced "deck" - like a "deck" of cards.

The LinkedList class is a pretty standard Deque and Queue implementation. It uses a linked list internally to model a queue or a deque.

The Java ArrayDeque class stores its elements internally in an array. If the number of elements exceeds the space in the array, a new array is allocated, and all elements moved over. In other words, the ArrayDeque grows as needed, even if it stores its elements in an array.

Deque Methods

Operation	Description	Example starting with deque: 59, 63, 19 (front is 59)
PushFront(deque, x)	Inserts x at the front of the deque	PushFront(deque, 41). Deque: 41, 59, 63, 19
PushBack(deque, x)	Inserts x at the back of the deque	PushBack(deque, 41). Deque: 59, 63, 19, 41
PopFront(deque)	Returns and removes item at front of deque	PopFront(deque) returns 59. Deque: 63, 19
PopBack(deque)	Returns and removes item at back of deque	PopBack(deque) returns 19. Deque: 59, 63
PeekFront(deque)	Returns but does not remove the item at the front of deque	PeekFront(deque) returns 59. Deque is still: 59, 63, 19
PeekBack(deque)	Returns but does not remove the item at the back of deque	PeekBack(deque) returns 19. Deque is still: 59, 63, 19
IsEmpty(deque)	Returns true if the deque is empty	IsEmpty(deque) returns false.
GetLength(deque)	Returns the number of items in the deque	GetLength(deque) returns 3.

Sets & Bags

The elements are stored in-random.

`Set` is ADT of the collection of elements which disallows duplicates,

`Bag` is ADT of the collection of elements which allows duplicates.

Bag

```
public class Bag<Item> implements Iterable<Item>
```

	`Bag()`	*create an empty bag*
`void`	`add(Item item)`	*add an item*
`boolean`	`isEmpty()`	*is the bag empty?*
`int`	`size()`	*number of items in the bag*

In-Class Activity

Write a pseudocode and draw diagrams for class Deque with the following methods:

implement using linked list

Operation	Description	Example starting with deque: 59, 63, 19 (front is 59)
PushFront(deque, x)	Inserts x at the front of the deque	PushFront(deque, 41). Deque: 41, 59, 63, 19
PushBack(deque, x)	Inserts x at the back of the deque	PushBack(deque, 41). Deque: 59, 63, 19, 41
PopFront(deque)	Returns and removes item at front of deque	PopFront(deque) returns 59. Deque: 63, 19
PopBack(deque)	Returns and removes item at back of deque	PopBack(deque) returns 19. Deque: 59, 63
PeekFront(deque)	Returns but does not remove the item at the front of deque	PeekFront(deque) returns 59. Deque is still: 59, 63, 19
PeekBack(deque)	Returns but does not remove the item at the back of deque	PeekBack(deque) returns 19. Deque is still: 59, 63, 19
IsEmpty(deque)	Returns true if the deque is empty	IsEmpty(deque) returns false.
GetLength(deque)	Returns the number of items in the deque	GetLength(deque) returns 3.